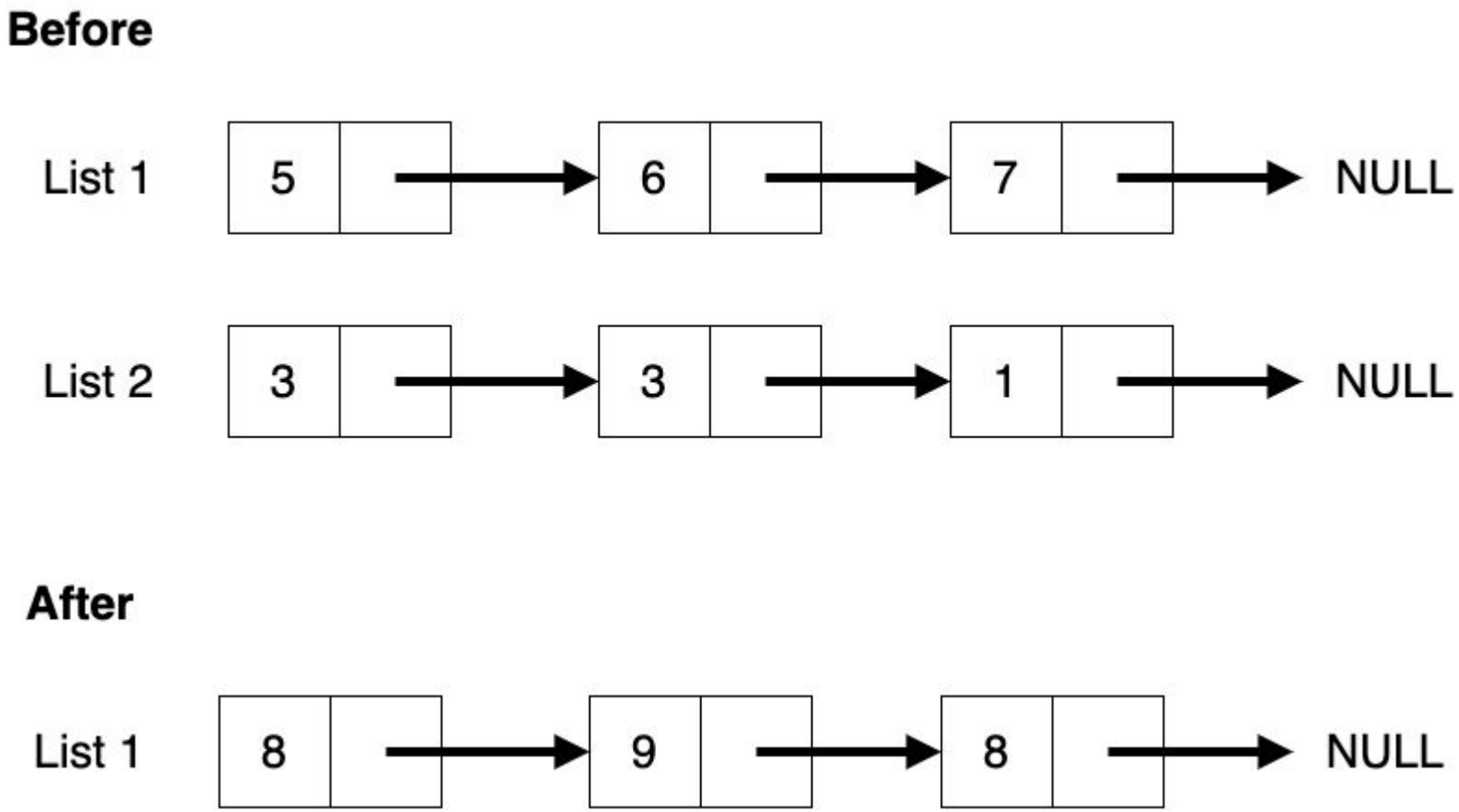
Practice Challenge Question Solutions

Kevin Browne

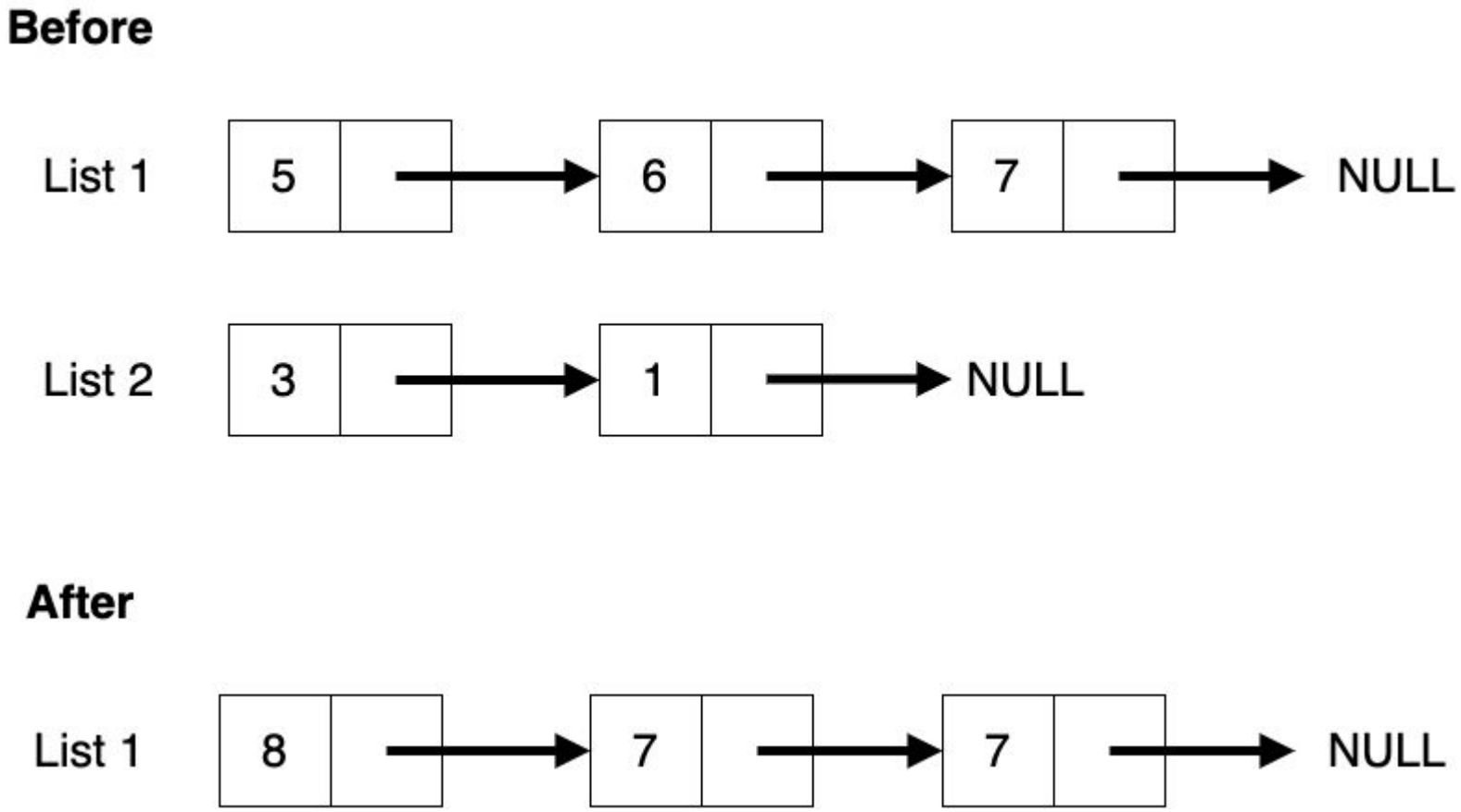
Add lists function

- The add lists function should accept pointers to two lists as an argument (list1 and list2)
- The value at the *ith* node in each list should be added together and the *ith* node in list1 should be set to the result
- If list1 or list2 are not the same length, than the adds should occur up until the shorter of the two list lengths

Add lists function example



Add lists function example



Add lists function

```
void add_lists(Node *list1, Node *list2)
{
  if (list1 == NULL || list2 == NULL) return ;
  list1->value = list1->value + list2->value;
  add_lists(list1->next, list2->next);
}
```

Testing the function with lists of equal length…

```
Node *list1, *list2, *list3, *list4, *list5, *list6;
list1 = list2 = list3 = list4 = list5 = list6 = NULL;
for (int i = 1; i <= 5; i++) list1 = insert_at_head(list1, i);
for (int i = 8; i >= 4 ; i--) list2 = insert_at_head(list2, i);
printf("\nList 1...\n");
print_list(list1);
printf("\nList 2...\n");
print_list(list2);
printf("\nList 1 after add...\n");
add_lists(list1, list2);
print_list(list1);
```

Test Results

```
List 1...
Node 0: 5
Node 1: 4
Node 2: 3
Node 3: 2
Node 4: 1

List 2...
Node 0: 4
Node 1: 5
Node 2: 6
Node 3: 7
Node 4: 8

List 1 after add...
Node 0: 9
Node 1: 9
Node 2: 9
Node 3: 9
Node 4: 9
```

Testing the function with lists of different length…

```
for (int i = 1; i <= 2; i++) list3 = insert_at_head(list3, i);
for (int i = 4; i <= 7; i++) list4 = insert_at_head(list4, i);
printf("\nList 3...\n");
print_list(list3);
printf("\nList 4...\n");
print_list(list4);
printf("\nList 3 after add...\n");
add_lists(list3, list4);
print_list(list3);
```

Test Results

```
List 3...
Node 0: 2
Node 1: 1

List 4...
Node 0: 7
Node 1: 6
Node 2: 5
Node 3: 4

List 3 after add...
Node 0: 9
Node 1: 7
```

Testing the function with lists of different length…

```
for (int i = 4; i <= 7; i++) list5 = insert_at_head(list5, i);
for (int i = 1; i <= 2; i++) list6 = insert_at_head(list6, i);
printf("\nList 5...\n");
print_list(list5);
printf("\nList 6...\n");
print_list(list6);
printf("\nList 5 after add...\n");
add_lists(list5, list6);
print_list(list5);
```

Test Results

```
List 5...
Node 0: 7
Node 1: 6
Node 2: 5
Node 3: 4

List 6...
Node 0: 2
Node 1: 1

List 5 after add...
Node 0: 9
Node 1: 7
Node 2: 5
Node 3: 4
```

Duplicate list function

- Create a function that when given a pointer to a linked list returns a pointer to a new linked list containing duplicate values
- The new list should be a different list created on the heap, but with the same number of nodes each with the same value as the original list
- Hint: recursively traverse the list, returning a pointer to the next node each time!

Duplicate list function

```
Node *duplicate_list(Node *node)
{
  if (node == NULL) return NULL;

  Node *new_node = calloc(sizeof(Node), 1);
  new_node->value = node->value;

  new_node->next = duplicate_list(node->next);

  return new_node;
}
```

Duplicate list test code

```
Node *list7 = NULL;
for (int i = 0; i < 10; i++) list7 = insert_at_head(list7, i);
printf("\nList 7...\n");
print_list(list7);
Node *list7_duplicate = duplicate_list(list7);
printf("\nList 7 duplicate...\n");
print_list(list7_duplicate);
```

Test Results

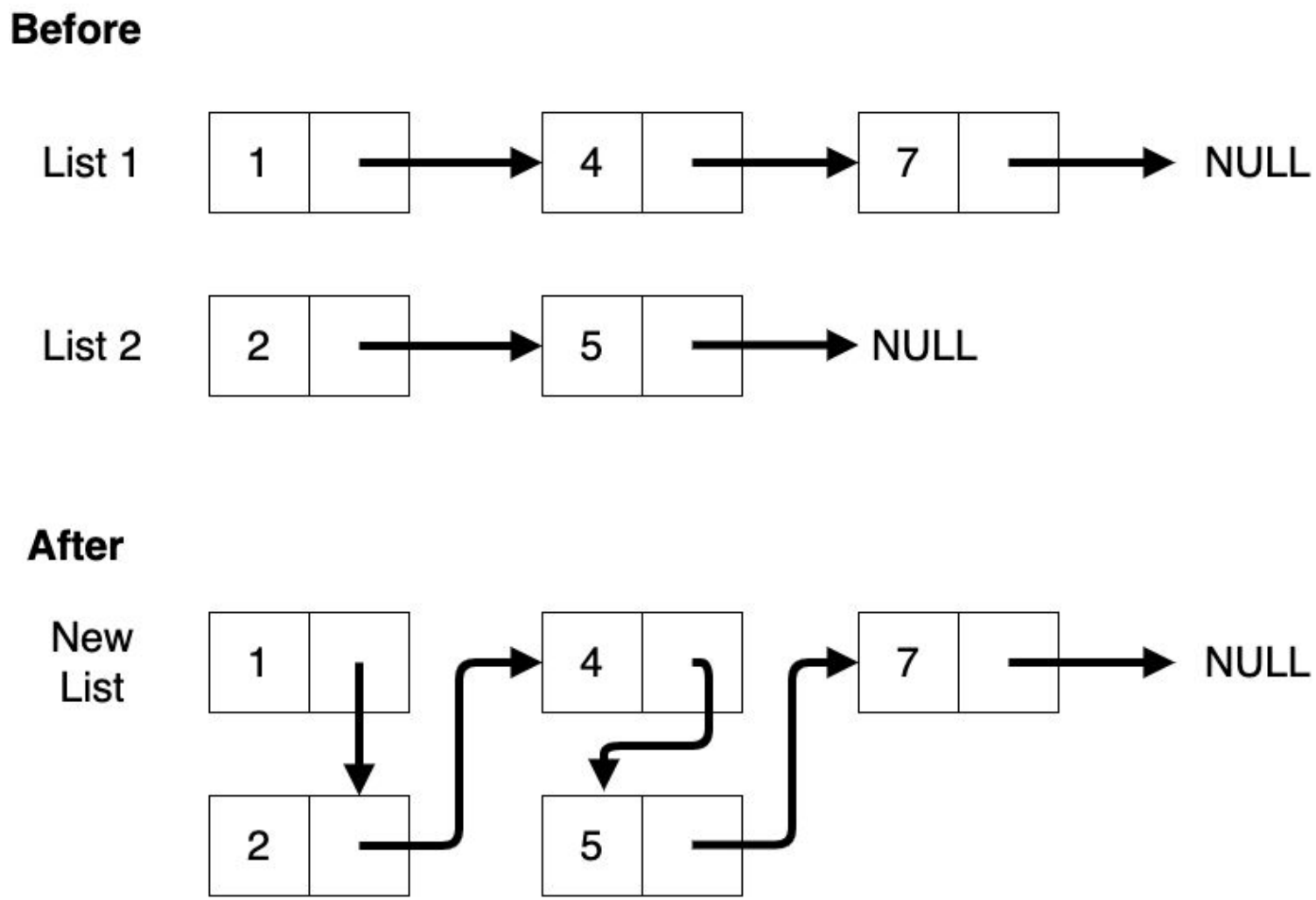
```
List 7...
Node 0: 9
Node 1: 8
Node 2: 7
Node 3: 6
Node 4: 5
Node 5: 4
Node 6: 3
Node 7: 2
Node 8: 1
Node 9: 0

List 7 duplicate...
Node 0: 9
Node 1: 8
Node 2: 7
Node 3: 6
Node 4: 5
Node 5: 4
Node 6: 3
Node 7: 2
Node 8: 1
Node 9: 0
```

Merge sorted list function

- Create a function that will accept pointers to two linked lists with node values sorted from lowest to highest

- The function should return a pointer to a merged list containing the same nodes on the heap as the original two linked lists, but now merged into a single list sorted from lowest to highest
  - If two nodes are equal in each list, just make a reasonable assumption (e.g. pick one list)

Merge sorted list example



If one list is empty, we simply return the other list…

```
Node *merge_sorted_lists(Node *list1, Node *list2)
{
  if (list1 == NULL) return list2;
  if (list2 == NULL) return list1;
```

Otherwise we determine the new head of the list, and maintain pointers to traverse both lists...

```
Node *l1current, *l2current;
Node *new_head, *newcurrent;

l1current = list1;
l2current = list2;

if (l1current->value <= l2current->value)
{
  new_head = l1current;
  l1current = l1current->next;
}
else
{
  new_head = l2current;
  l2current = l2current->next;
}
newcurrent = new_head;
```

Traverse both lists, re-arranging nodes to point to each other based on the order of their values...

```
while (l1current != NULL && l2current != NULL)
{
  if (l1current->value <= l2current->value)
  {
    newcurrent->next = l1current;
    newcurrent = l1current;
    l1current = l1current->next;
  }
  else
  {
    newcurrent->next = l2current;
    newcurrent = l2current;
    l2current = l2current->next;
  }
}
```

Once we reach the tail of a list, we can append what remains of the other list (it may be NULL too)

```
    if (l1current == NULL)
    {
      newcurrent->next = l2current;
    }
    else if (l2current == NULL)
    {
      newcurrent->next = l1current;
    }
    return new_head;
}
```

Test the function by creating two sorted lists of length 10 with values from 0-20…

```
srand(time(NULL));
Node *list11 = NULL;
Node *list12 = NULL;
for (int i = 0; i < 10; i++) list11 = insert_at_head(list11, rand() % 99);
for (int i = 0; i < 10; i++) list12 = insert_at_head(list12, rand() % 99);
sort_list(list11);
sort_list(list12);
printf("\nList 11...\n");
print_list(list11);
printf("\nList 12...\n");
print_list(list12);
Node *newhead = merge_sorted_lists(list11, list12);
printf("\nMerged list...\n");
print_list(newhead);
```

Test Result: Input Lists

```
List 11...
Node 0: 6
Node 1: 18
Node 2: 22
Node 3: 25
Node 4: 54
Node 5: 63
Node 6: 65
Node 7: 74
Node 8: 83
Node 9: 86

List 12...
Node 0: 22
Node 1: 23
Node 2: 35
Node 3: 41
Node 4: 42
Node 5: 63
Node 6: 90
Node 7: 95
Node 8: 97
```

Test Result: Merged List

```
Merged list...
Node 0: 6
Node 1: 18
Node 2: 22
Node 3: 22
Node 4: 23
Node 5: 25
Node 6: 35
Node 7: 41
Node 8: 42
Node 9: 54
Node 10: 63
Node 11: 63
Node 12: 65
Node 13: 74
Node 14: 83
Node 15: 86
Node 16: 90
Node 17: 95
Node 18: 97
Node 19: 97
```